Kubernetes and Istio

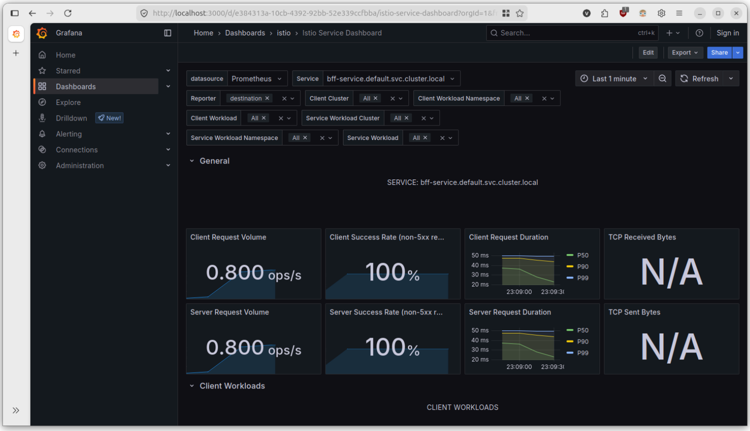
10 - Istio observability

Generated: June 2026
Version 1.0

10 - Istio observability

In a microservices architecture, tracking requests as they traverse multiple network boundaries is notoriously difficult. Istio solves this by injecting Envoy sidecar proxies next to your workloads. These proxies intercept all inbound and outbound traffic, allowing Istio to automatically generate rich telemetry-metrics, logs, and distributed traces-without requiring you to change your application code. Two of the most critical tools in the Istio ecosystem for visualizing this telemetry are **Grafana** (for *performance metrics*) and **Kiali** (for *usage metrics* and *topology*).

1. Grafana


http://localhost:3000/d/e384313a-10cb-4392-92bb-52e339ccfbba/istio-service-dashboard?orgId=1&f
Grafana
Home
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Connections
Administration
Home › Dashboards › istio › Istio Service Dashboard
Search...
ctrl+k
Sign in
Edit
Export
Share
Last 1 minute
Refresh
General
Client Request Volume
0.800 ops/s
Client Success Rate (non-5xx re...
100%
Client Request Duration
50 ms
40 ms
30 ms
20 ms
23:09:00
23:09:3
P50
P90
P99
TCP Received Bytes
N/A
Server Request Volume
0.800 ops/s
Server Success Rate (non-5xx r...
100%
Server Request Duration
50 ms
40 ms
30 ms
20 ms
23:09:00
23:09:3
P50
P90
P99
TCP Sent Bytes
N/A
Client Workloads
CLIENT WORKLOADS

datasource: Prometheus
Service: bff-service.default.svc.cluster.local
Reporter: destination
Client Cluster: All
Client Workload Namespace: All
Client Workload: All
Service Workload Cluster: All
Service Workload Namespace: All
Service Workload: All

SERVICE: bff-service.default.svc.cluster.local

Grafana is an open-source analytics and interactive visualization web application. In the context of Istio, it is used to query and visualize the time-series metrics collected by Prometheus from the Envoy proxies.

> 🗒️ ***Installation***
> *Follow the guide at 8 - Istio # Run Grafana.*

What kind of metrics you can find

Istio exports standard metrics out-of-the-box that map directly to the Four Golden Signals of monitoring:

- **Traffic** (Volume): Measures the total number of requests (istio_requests_total).
- **Latency**: Measures the duration of requests (istio_request_duration_milliseconds).
- **Errors**: Tracks failed requests, usually segmented by HTTP status codes (e.g., 4xx and 5xx errors).
- **Saturation**: Tracks how "full" the service is (e.g., TCP connections, proxy CPU/memory usage).

When you install Grafana with Istio, you get several pre-configured dashboards:

- **Mesh Dashboard**: A high-level overview of the entire service mesh, showing global success rates and request volumes.
- **Service Dashboard**: Drills down into individual services (e.g., evaluating the performance of the checkout service).
- **Workload Dashboard**: Focuses on the underlying deployments or pods backing a service.

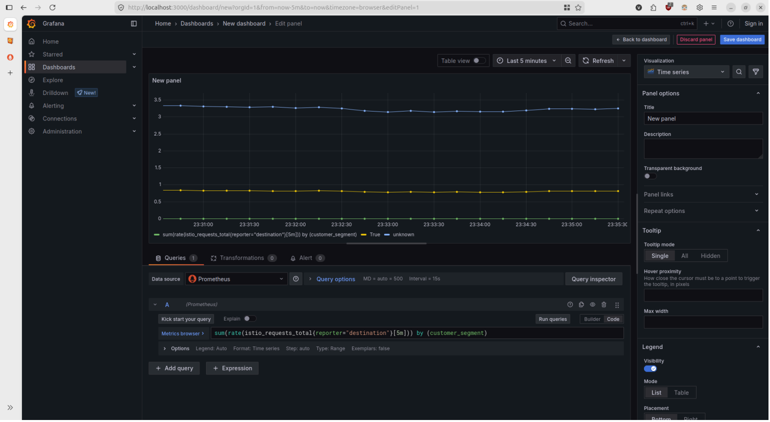
Add custom metrics

While standard metrics are powerful, you often need business-specific or infrastructure-specific dimensions (like tracking requests by a specific customer segment or custom HTTP header). Istio allows you to add custom metrics using the `Telemetry API`. You configure this by creating a Telemetry Custom Resource Definition (CRD). The following example demonstrates how to extract a custom header (`x-beta-version`) and append it as a new dimension to your metrics:

```
apiVersion: telemetry.istio.io/v1alpha1
kind: Telemetry
metadata:
  name: custom-metrics
  namespace: istio-system
spec:
  metrics:
    - providers:
        - name: prometheus
      overrides:
        - match:
            metric: REQUEST_COUNT
          tagOverrides:
            beta_users:
              value: "request.headers['x-beta-version']"
```

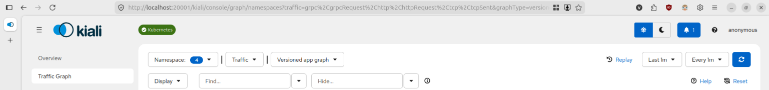
Then add the following PromQL query to the dashboard:

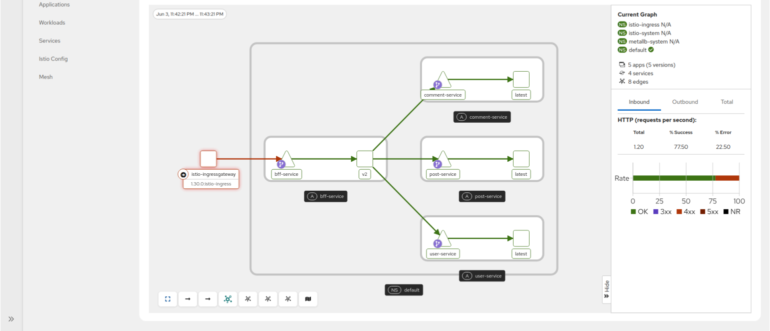
```
sum(rate(istio_requests_total{reporter="destination"}[5m])) by (customer_segment)
```


http://localhost:3000/dashboard/new?orgId=1&from=now-5m&to=now&timezone=browser&editPanel=1
Grafana
Home › Dashboards › New dashboard › Edit panel
Search... ctrl+k
Sign in
Back to dashboard
Discard panel
Save dashboard
Home
Starred
Dashboards
Explore
Drilldown New!
Alerting
Connections
Administration
Table view
Last 5 minutes
Refresh
New panel
3.5
3
2.5
2
1.5
1
0.5
0
23:31:00
23:31:30
23:32:00
23:32:30
23:33:00
23:33:30
23:34:00
23:34:30
23:35:00
23:35:30
sum(rate(istio_requests_total{reporter="destination"}[5m])) by (customer_segment)
True
unknown
Queries 1
Transformations 0
Alert 0
Data source
Prometheus
Query options MD = auto = 500 Interval = 15s
Query inspector
A (Prometheus)
Kick start your query
Explain
Run queries
Builder Code
Metrics browser ›
Options Legend: Auto Format: Time series Step: auto Type: Range Exemplars: false
Add query
Expression
Visualization
Time series
Panel options
Title
New panel
Description
Transparent background
Panel links
Repeat options
Tooltip
Tooltip mode
Single All Hidden
Hover proximity
How close the cursor must be to a point to trigger the tooltip, in pixels
Max width
Legend
Visibility
Mode
List Table
Placement
Bottom Right

```
sum(rate(istio_requests_total{reporter="destination"}[5m])) by (customer_segment)
```

2. Kiali


http://localhost:20001/kiali/console/graph/namespaces?traffic=grpc%2CgrpcRequest%2Chttp%2ChttpRequest%2Ctcp%2CtcpSent&graphType=versior
kiali
Kubernetes
anonymous
Overview
Traffic Graph
Namespace: 4
Traffic
Versioned app graph
Replay
Last 1m
Every 1m
Display
Find...
Hide...
Help
Reset


Applications
Workloads
Services
Istio Config
Mesh
Jun 3, 11:42:21 PM ... 11:43:21 PM
istio-ingressgateway
1.30.0:istio-ingress
bff-service
v2
comment-service
latest
post-service
latest
user-service
latest
A bff-service
A comment-service
A post-service
A user-service
NS default
Hide
Current Graph
NS istio-ingress N/A
NS istio-system N/A
NS metallb-system N/A
NS default
5 apps (5 versions)
4 services
8 edges
Inbound
Outbound
Total
Rate
0
25
50
75
100
OK
3xx
4xx
5xx
NR

HTTP (requests per second):

Total	% Success	% Error
1.20	77.50	22.50

While Grafana excels at rendering time-series data as graphs and gauges, Kiali struggles to answer questions like: "*Which services are talking to each other?*" or "*Where is the traffic bottleneck in this specific transaction?*". Kiali is a management console specifically built for Istio to visualize service mesh topology and configuration health.

> 🗒️ ***Installation***
> *Run the following commands from the root of your downloaded Istio directory:*
>
> ```
> kubectl apply -f samples/addons/prometheus.yaml
> kubectl apply -f samples/addons/kiali.yaml
> ```
>
> *Once installed access the UI by port-forwarding.*
>
> ```
> istioctl dashboard kiali
> ```

What you can find

Kiali infers the structure of your microservices by analyzing the traffic flowing through them. It provides:

- **Topology Mapping (Service Graphs)**: Kiali auto-generates visual graphs of your architecture (App, Versioned App, Workload, and Service views). If a service starts failing, the corresponding node and its incoming edges turn red, allowing for immediate root-cause isolation.
- **Traffic Distribution & Shifting**: If you are running multiple versions of a service (e.g., v1 and v2), Kiali visually displays the exact percentage of traffic going to each. It even provides a wizard to implement traffic splitting directly from the UI.
- **Configuration Validation**: Kiali scans your Istio configuration objects (VirtualServices, DestinationRules, Gateways) and flags misconfigurations or orphaned routes before they cause an outage.
- **Security Visualization**: It visually indicates which edges are secured with mTLS (Mutual TLS) by displaying padlock icons on the connection lines.

> ❓ ***Why it is useful***
> *Modern microservice environments are too complex to map manually. Kiali bridges the gap between infrastructure configuration and real-time observability, turning raw Prometheus metrics into an actionable, live map of your distributed system.*